

LangChain Agent

Agentic AI Python Tutor — Sprint 1



LLM wünscht — LangChain erfüllt



AGENT



LLM



TOOLS

AGENDA

Was haben wir **gebaut**?

01 — INTERFACE

UI zur Interaktion

Next.js Frontend — Student schreibt Code, stellt Fragen und sieht sofort die Antwort des Agents.



02 — BACKEND

FastAPI Endpoint

Empfängt Requests vom Frontend und leitet sie an den Agent weiter. Endpunkte: */analyze*, */chat*, */run*



03 — KERN

LangChain Agent

Orchestriert den Tool-Loop — LLM entscheidet selbst welche Tools aufgerufen werden bis die Antwort fertig ist.



04 — TOOLS

Spezialisierte Tools

3 Tools: *explain* · *debug* · *exercise* — jedes mit eigenem LLM-Prompt.



05 — VERGLEICH

GPT vs. Mein Agent

GPT antwortet mit Text — unser Agent plant, wählt Tools und liefert *strukturierte* Ergebnisse.



06 — DEMO

Live Demo

Direkt ausprobieren — *agentic-ai-python-tutor.vercel.app*

FRONTEND — KURZ

Was der Student sieht und macht

FRONTEND — WAS DER STUDENT SIEHT

**Code-Editor**

Python-Code schreiben und direkt im Browser ausführen

**Code analysieren**

Ein Klick — Agent erklärt, findet Fehler, gibt Übung

**Chat mit dem Tutor**

Fragen stellen mit vollem Code-Kontext — nur Python-Fragen erlaubt

**Code ausführen**

Python-Code wird sicher im Backend ausgeführt, Output sofort angezeigt

**„In Editor“ Button**

Python-Code aus Chat-Antworten per Klick in den Editor übernehmen und sofort ausführen

BACKEND — API ENDPUNKTE

POST `/tutor/analyze`

Code → Agent → strukturierte Antwort mit Erklärung, Fehler & Übung

POST `/tutor/chat`

Frage + Code + Verlauf → kontextbewusste Antwort vom Agent

POST `/tutor/run`

Code sicher ausführen → stdout / stderr direkt zurück

KERN DER ARCHITEKTUR

LangChain Agent — wie er denkt

AGENT LOOP — WIE ER DENKT



DIE 3 TOOLS DES AGENTS

**explain_code_tool**

agent/tools/explain_tool.py

Erklärt den Code auf Deutsch in 3–4 Sätzen. Was macht er, wie funktioniert er, einfache Sprache für Anfänger.

ERKLÄRUNG

**debug_code_tool**

agent/tools/debug_tool.py

Analysiert den Code auf Syntax- und Logikfehler. Gibt Fehlertyp und konkreten Verbesserungsvorschlag zurück.

FEHLER FINDEN

**exercise_tool**

agent/tools/exercise_tool.py

Generiert eine passende Übungsaufgabe basierend auf dem Code des Studenten — maximal 3 Sätze, direkt anwendbar.

ÜBUNG

WARUM KEIN CHATBOT?

Agent vs. ChatGPT / Chatbot



ChatGPT / Chatbot

Reagiert auf Text — antwortet mit Text

🧠 VERARBEITUNG

LLM generiert Antwort **direkt** — kein Werkzeug, keine Zwischenschritte

🔄 LOOP

Keiner — eine Frage, eine Antwort, fertig

📄 AUSGABE

Ein freier Textblock — keine feste Struktur, keine spezialisierten Aktionen



Unser LangChain Agent

Plant, handelt, überprüft — eigenständig

🧠 VERARBEITUNG

LLM entscheidet welche **Tools** gebraucht werden — explain · debug · exercise

🔄 LOOP

Läuft bis `tool_calls` leer ist — mehrere Runden möglich

📄 AUSGABE

5 strukturierte Felder: Erklärung · Fehler gefunden · Fehlertyp · Hinweis · Übung

VS

AUSBlick

Next Sprint — Was kommt als nächstes?



SPRINT 2 — FEATURE

RAG + PDF Upload

Student lädt eigenes Kursmaterial als PDF hoch. Der Agent durchsucht es semantisch — bei jeder Analyse und im Chat wird das Material als Kontext genutzt.

- PDF → Chunks → FAISS Vector Store
- Seitenreferenz „Seite 9“ direkt abrufbar
- Semantische + direkte Suche kombiniert



SPRINT 2 — FEATURE

Lernfortschritt-Tracking

Der Tutor merkt sich welche Konzepte der Student bereits geübt hat. Übungen werden gezielt auf Schwächen zugeschnitten — kein Wiederholen was schon sitzt.

- Session-Historie: gelöste Aufgaben
- Schwierigkeitsgrad steigt automatisch
- Fortschrittsanzeige im Frontend



SPRINT 2 — FEATURE

Schülerprofil

Jeder Student bekommt ein persistentes Profil — Lernstand, Schwächen, bevorzugte Erklärungsstile. Der Agent passt Ton und Tiefe automatisch an den Schüler an.

- Login / Profil-Persistenz
- Lernstil-Erkennung aus Verlauf
- Personalisierte Übungen & Erklärungen

Live Demo